

Movie Ticket Booking System

Functional Requirements

Requirement	Description
Movie Listing	Show available movies to customer with details like Movie Name, Language, Time of the show.
Seat Layout	Shows the customer seat layout with available seats. Layout: Booked Seat -> [X], while Available Seat -> []. Seats are categorized into three levels like PLATINIUM(Rs.400), GOLD(250), SILVER(Rs.150) with different prices.
Booking & Pricing	Customer initiates the booking process by selecting the desired ticket level, number of tickets and calculates the total bill and then proceeds to payment section for confirmation for the tickets.
Payment Processing	Payment can be done by CASH, UPI, CARD. If the payment is successful then the receipt is generated, other payment failed and the selected seat's status turns to AVAILABLE instead of BOOKED. (if the selected seat is already booked then the booking is CANCELLED by default).
Cancellation	A customer can cancel a booking by entering the unique booking id number, then the status updates to CANCELLED and all the seats gets released which are associated with the booking id number.

Non-Functional Requirements

Requirement	Description
Modularity	The system must maintain strict modularity, organizing code cleanly with one class per file and no header files.
Extensibility	The system must follow the Open/Closed Principle so that adding a new payment method (such as NetBanking) does not require modifying existing booking or service classes.
Input Validation	The system must handle invalid inputs (such as non-existent seat numbers, invalid menu choices, or malformed strings) with clear error prompts without crashing the program.

Requirement	Description
Transactional Integrity	Seat reservations must be atomic: all selected seats are either successfully confirmed upon payment or released back to available status if an error or failure occurs

Step B: Noun-Verb Analysis

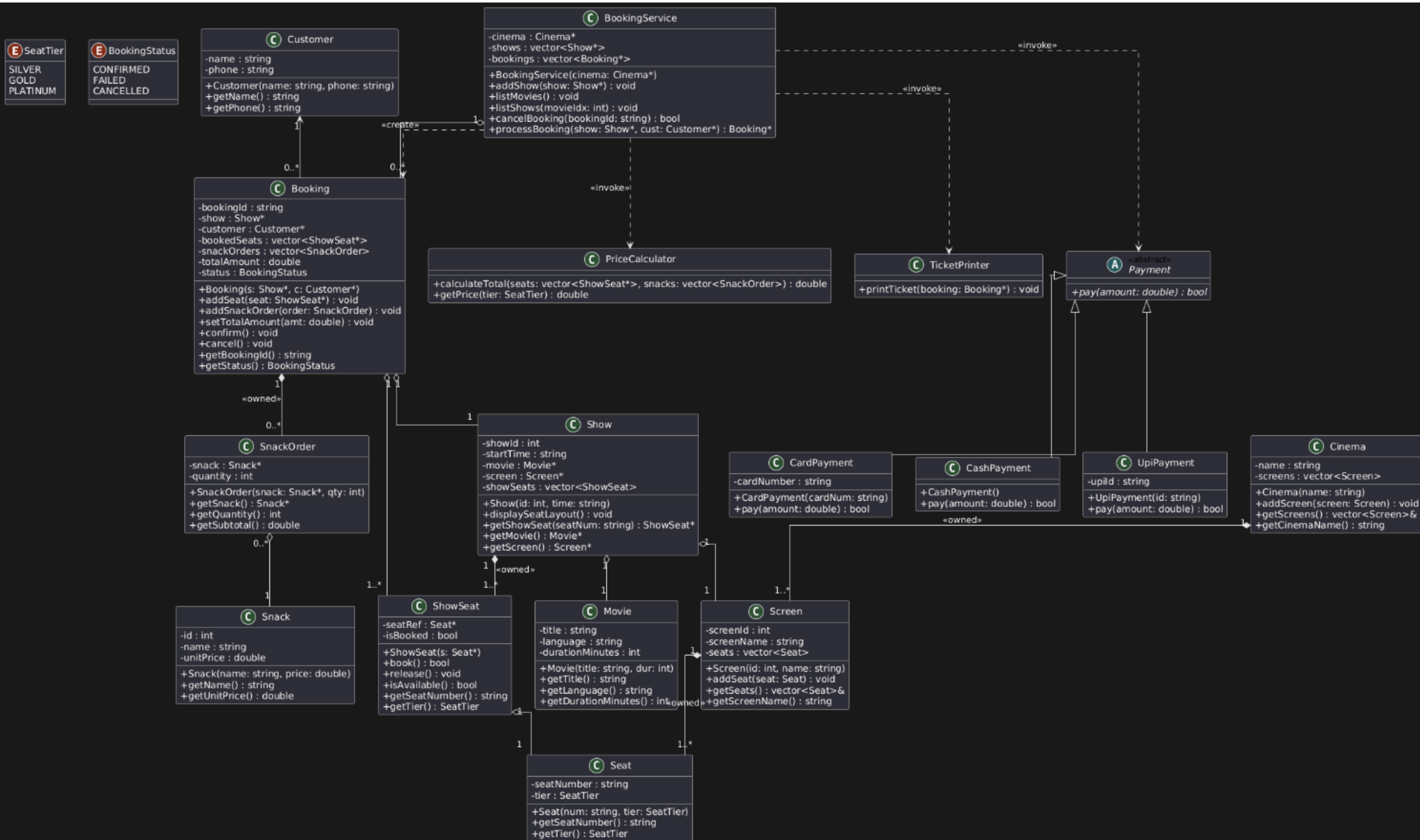
Noun Found	Keep as a Class?	Reason / Justification
Movie	Yes	Has its own data and identity (title, language, duration).
Seat	Yes	Represents a physical chair with a seat number and tier.
"seat layout"	No	It is a dynamic view of a Show's seats, not a standalone entity; handled via a display method.
Screen	Yes	Represents an auditorium that physically owns a collection of seats.
Cinema	Yes	The multiplex venue that owns its screens.
Show	Yes	Binds a Movie to a Screen at a specific time; owns its ShowSeats.
ShowSeat	Yes	Tracks the status of one seat for one show (AVAILABLE/BOOKED).
Customer	Yes	Captures customer identity (name and phone number).
"duration"	No	It is a primitive attribute (<code>int durationMinutes</code>) inside the <code>Movie</code> class, not an object.
Snack	Yes	Catalog item for concessions (name, unit price) offered optionally at checkout.
SnackOrder	Yes	Pairs a selected snack item with a user-specified quantity in the cart.

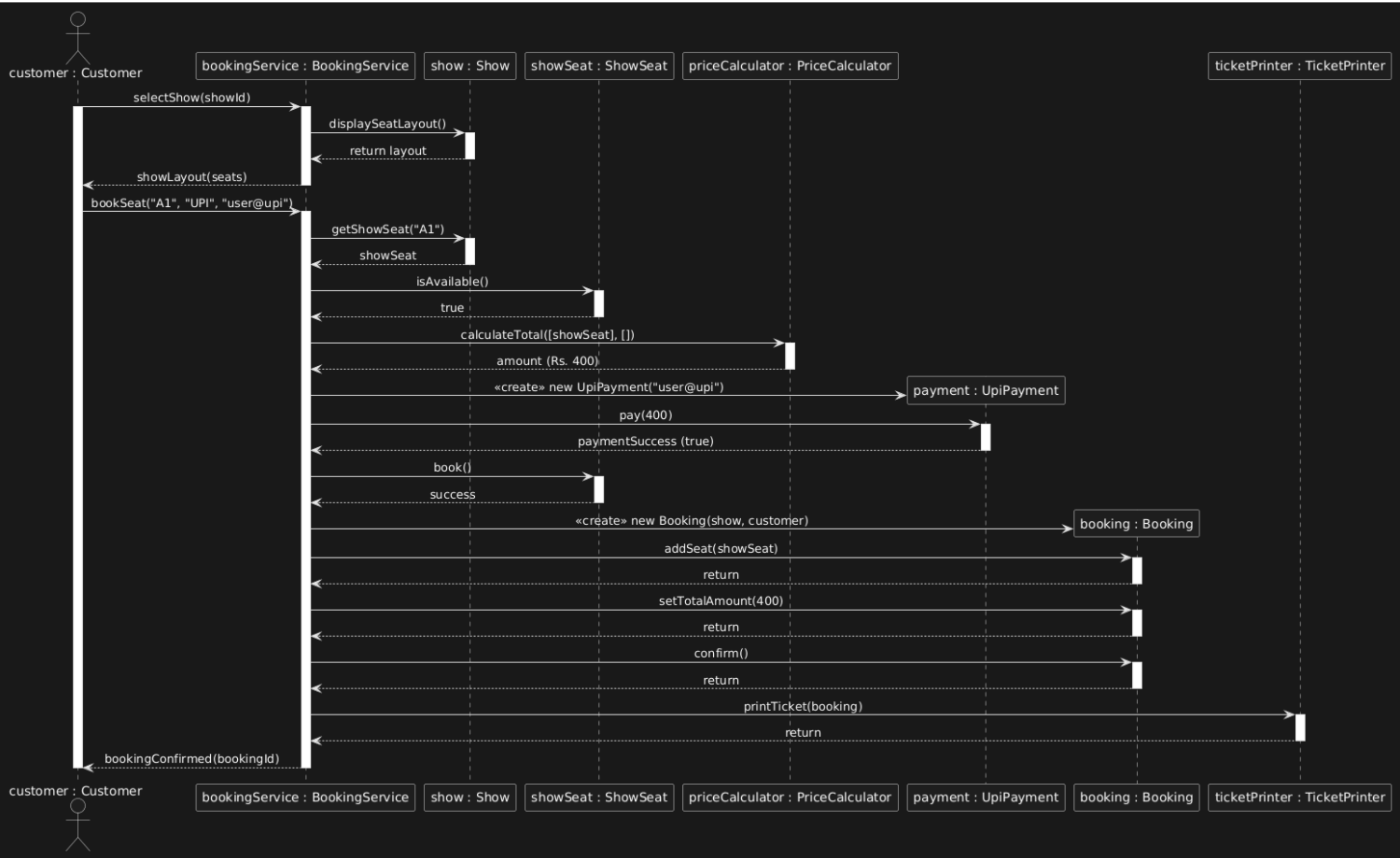
Noun Found	Keep as a Class?	Reason / Justification
"seat tier" / "tier price"	No	Represented as an enumeration (<code>enum SeatTier</code>) and constant values, not a full class.
Booking	Yes	Core transaction holding booking ID, show, seats, snacks, amount, and status.
"booking id"	No	It is an identifier attribute within the <code>Booking</code> class, not an independent entity.
Payment	Yes (Abstract)	Defines the payment contract interface (<code>pay(amount)</code> pure virtual).
UPI / Card / Cash	Yes	Concrete payment strategies realizing distinct checkout flows.
"receipt" / "ticket copy"	No	It is formatted text output generated on standard output, not a stateful domain entity.
Price Calculator	Yes	Service class dedicated to calculating total bill from seat tiers and snacks.
Ticket Printer	Yes	Utility class strictly responsible for printing formatted tickets to the console.
Booking Service	Yes	The orchestrator running the booking and cancellation flow end to end.
"menu"	No	Transient presentation loop in the console interface (<code>main.cpp</code>), not a business entity.

Relationship Table

Pair	Relationship	Lifetime Test Justification
Cinema — Screen	Composition	A <code>Cinema</code> creates and physically owns its <code>Screen</code> objects. If the cinema building is destroyed, its screens cease to exist.
Screen — Seat	Composition	A <code>Screen</code> owns its physical chairs (<code>Seat</code>). If a screen auditorium is torn down or dismantled, those physical seats belonging to that hall are destroyed.
Show — Movie	Aggregation	A <code>Show</code> borrows a <code>Movie</code> . If the 6:00 PM show is canceled or removed from memory, the movie itself still exists in the catalog and can play at 9:00 PM.
Show — Screen	Aggregation	A <code>Show</code> references an existing <code>Screen</code> auditorium. If the show ends or is deleted, the physical auditorium remains intact.
Show — ShowSeat	Composition	A <code>ShowSeat</code> represents status specifically for that individual screening. If the <code>Show</code> is deleted, its show-specific seat availability records die with it.
Booking — Customer	Association	A <code>Booking</code> is initiated by a <code>Customer</code> . Deleting or canceling an order does not destroy the customer profile.
Booking — ShowSeat	Aggregation	A <code>Booking</code> keeps pointers to reserved <code>ShowSeat</code> items. If the booking is canceled or removed, the show seats are not destroyed—they simply revert back to AVAILABLE.
Booking — SnackOrder	Composition	A <code>SnackOrder</code> is a custom line-item belonging strictly to that single booking. If the booking transaction is canceled or destroyed, that order line ceases to exist.
SnackOrder — Snack	Aggregation	A <code>SnackOrder</code> holds a pointer to an item from the cinema's concession menu (<code>Snack</code>). Deleting a user's

Pair	Relationship	Lifetime Test Justification
		snack order does not remove the item from the catalog.
Booking — Payment	Association	<code>Booking</code> uses <code>Payment</code> during the checkout transaction via method injection. Neither object owns the lifetime of the other.
Payment — UpiPayment	Inheritance	<code>UpiPayment</code> is an <code>is-a</code> specialized realization of the abstract base <code>Payment</code> contract.
BookingService — Booking	Aggregation	<code>BookingService</code> maintains a collection of active <code>Booking</code> objects. Deleting the service does not alter the conceptual transaction history of the cinema.





SOLID Mapping

- **S (SRP):** PriceCalculator handles only math, TicketPrinter handles only display, and BookingService orchestrates the workflow.
- **O (OCP):** New payment methods extend Payment without altering existing checkout code.
- **L (LSP):** UpiPayment, CardPayment, and CashPayment substitute Payment* interchangeably without breaking behavior.
- **I (ISP):** Payment interface is lean—exposing only pay(), with no bloated, unused methods.
- **D (DIP):** BookingService depends on the abstraction Payment*, not concrete payment classes.

One Thing Deliberately NOT Done

- **Did NOT combine Seat and ShowSeat:** Seat represents immutable hardware (Audi row/number), while ShowSeat holds show-specific state (isBooked). Merging them would prevent multiple shows from sharing the same physical seats.

```
C:\Users\ACER\Desktop\MovieTicket System\Modular Code>g++ main.cpp && a
```

```
=====
MOVIE TICKET BOOKING SYSTEM
=====
```

```
=== Seat Layout for Spider-Man: Brand New Day (07:00 PM) ===
[A1 : AVAILABLE] [A2 : AVAILABLE] [B1 : AVAILABLE]
=====
```

```
>>> Booking seat A1 via UPI...
Authorized Rs. 400 from VPA: tajinder@okaxis
```

```
=====
TICKET RECEIPT
=====
```

```
Booking ID : BMS-1001
Customer   : Tajinder Singh (+91-1234567890)
Movie      : Spider-Man: Brand New Day
Time       : 07:00 PM
Auditorium : Audi-01 (IMAX)
Seats      : A1
Snacks     : None
Total Paid : Rs. 400.00
Status     : CONFIRMED
=====
```



```
>>> Attempting to book already BOOKED seat A1...
Seat A1 is already BOOKED. No changes made.
```

```
>>> Attempting to book seat A2 with failing payment...
Payment timed out/failed for UPI ID: tajinder@badupi
Transaction could not be completed. Seat A2 remains AVAILABLE.
```

```
=== Seat Layout for Spider-Man: Brand New Day (07:00 PM) ===
[A1 : BOOKED] [A2 : AVAILABLE] [B1 : AVAILABLE]
```

```
=====
```

```
>>> Cancelling booking BMS-1001...
Booking BMS-1001 cancelled. Associated seats are now AVAILABLE again.
```

```
=== Seat Layout for Spider-Man: Brand New Day (07:00 PM) ===
[A1 : AVAILABLE] [A2 : AVAILABLE] [B1 : AVAILABLE]
```

```
=====
```

```
>>>Selecting non-existent seat A21...
Seat "A21" does not exist in this auditorium!
```

```
>>> Selecting invalid show index 5...
Invalid show selection.
```